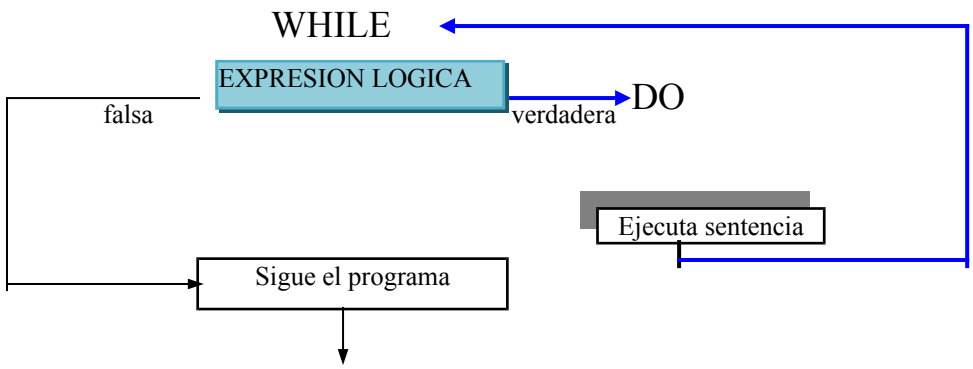
Capítulo 4

Estructura de control: Repetición

1. Introducción

Las **sentencias de repetición** especifican que ciertas sentencias deben ejecutarse en forma repetitiva. Estas sentencias también se denominan **ciclos.** Si el número de repeticiones no se conoce a-priori (antes que las repeticiones comiencen) la construcción mas apropiada es WHILE o REPEAT. En caso contrario, se recomienda FOR.

2. Estructura WHILE



WHILE verifica el valor de verdad de una expresión lógica. Si el resultado es verdadero, ejecuta la/s sentencia/s, y vuelve a verificar la expresión. Mientras sea verdadera, sigue ejecutando la/s sentencia/s. Cuando es falsa, termina el ciclo, y el control de ejecución sigue en la sentencia siguiente a WHILE. Si la sentencia es simple, la sintaxis es:

WHILE expresión lógica **DO**
Sentencia;

Si la sentencia es compuesta la sintaxis correspondiente es:

WHILE expresión lógica **DO**
begin
$Sentencia_1$;
$Sentencia_2$;
.....;
$Sentencia_n$
end;

Observaciones:

- Notar que, si al iniciarse el ciclo WHILE la expresión lógica es falsa, no se ejecuta la/s sentencia/s.
- En la expresión lógica debe figurar alguna/s variable/s que la modifiquen durante la ejecución del ciclo.

2.1 Ciclo While

La **expresión lógica** cuyo resultado determina la continuación (true) o la finalización del ciclo (false) puede presentarse de diferentes formas.
Generalmente se utilizan operadores relacionales para enfrentar una variable a un valor preestablecido.
Algunos ejemplos:
- Si el ciclo debe repetirse una determinada cantidad de veces, se requiere evaluar si un contador de repeticiones ha alcanzado dicha cantidad.
- Otra posibilidad es que desconociendo la cantidad de repeticiones que se deben ejecutar, el ciclo debe detenerse cuando una variable toma por lectura un valor "imposible" o por cálculo un resultado límite.
Estas situaciones pueden combinarse mediante el uso de operadores lógicos, que potencian el control que se ejerce sobre el ciclo.

Veremos a continuación algunos ejemplos

E1) Luego de leer un conjunto de números cuya cantidad no se conoce de antemano (se sabe que son distintos de cero), escribir los que sean mayores que 23.

```
Program E1;
 Var
    num: real;
 Begin
 WRITE('Ingresar un numero:');
 READLN(num);
 WHILE num <> 0 DO
       Begin
       IF num>23 THEN
           WRITELN(num);
       WRITE('Ingresar un numero:');
       READLN(num);
       End
End.
```

E2) En un lote de números positivos, cuya cantidad no se conoce a-priori, contar cuantos números hay entre 100 y 200.

```
Program E2;
 Var
    Contador:integer;
    num:real;
 Begin
 Contador:=0;
 WRITE('Ingresar un numero:');
 READLN(num);
 WHILE num>0 DO
      Begin
      IF (num>=100) AND (num<=200) THEN
                Contador:=Contador+1;
      WRITE('Ingresar un numero:');
      READLN(num)
      End;
  WRITELN('La cantidad de datos es ',Contador)
  End.
```

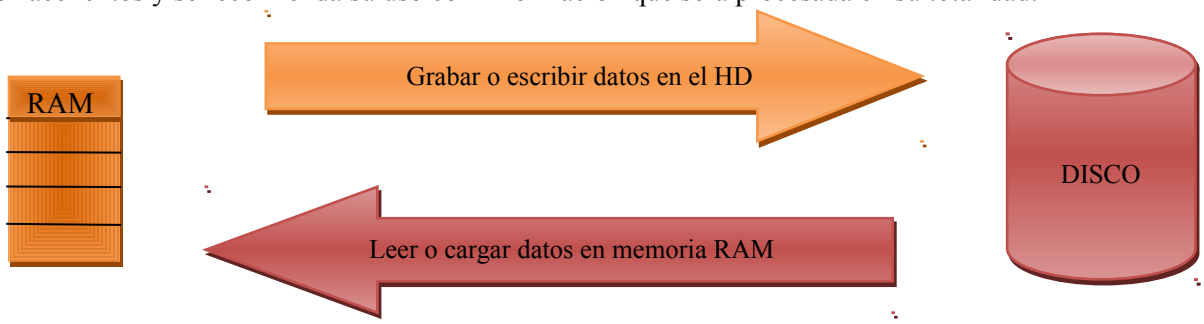
E3) Ingresar números y sumarlos mientras dicha suma sea menor que un tope. Escribir el resultado de la suma.

```
program E3;
 var
   Tope,suma,A:integer;
 begin
 writeln('ingrese un Tope para la suma');
 readln(Tope);
 suma:=0;
 WHILE suma<Tope DO
     begin
     readln(A);
     suma:=suma+A;
     end;
 writeln('El valor de suma es:',suma);
 end.
```

2.1.5 Ciclo de lectura de datos desde archivo

Tanto la entrada como la salida estándar, usan los archivos del sistema ***input*** y ***output*** por defecto, o sea que no es necesario especificar el teclado y la pantalla respectivamente.
Si se requiere lectura o escritura sobre archivo es necesario indicar el nombre del archivo (y en el caso de que este no se encuentre en la misma ubicación del programa, también indicar unidad y carpeta que lo contiene), como así también la operación a realizar sobre el mismo (leer o escribir).

Consideraciones previas para lectura o escritura desde un almacenamiento secundario con archivos de texto (*text*)
Si el programador desea ingresar datos desde el disco rígido o removible (donde fueron almacenados previamente como un ***archivo de texto***), debe indicar al programa el nombre del archivo que contiene los datos y su intención de leer datos desde ese archivo. De manera similar, puede crearse y almacenar datos en un archivo de texto. Los archivos de texto en Pascal requieren de variables de tipo ***text*** . Estos archivos son secuenciales, es decir se acceden en forma progresiva desde el comienzo al final, esta característica los hace lentos y se recomienda su uso con información que será procesada en su totalidad.



El programa Pascal se ejecuta desde la memoria de acceso aleatorio (o RAM). Si el programador desea almacenar datos en un archivo de texto en el disco, estos deben ser grabados con sentencias específicas de escritura. Por otro lado, si desea leer o cargar datos ya previamente almacenados en el disco para llevarlos a la memoria, deben ser recuperados con otras sentencias específicas. Un archivo de texto es utilizado para leer o grabar, en forma excluyente o sea no admite ambas operaciones simultáneamente.

Los datos en un archivo de texto se almacenan usando el código ASCII, en el cual cada carácter es representado por un simple byte. Debido a que los archivos de texto utilizan el código ASCII, se pueden ver o imprimir. En este tipo de archivos, todos sus datos se almacenan como cadenas de caracteres, es decir, los números se almacenan con su representación ASCII y no su representación numérica, por lo tanto no se pueden realizar operaciones matemáticas directamente con ellos. Pero, si el dato es leído desde disco y almacenado en una variable numérica, puede operarse normalmente.

Luego de declarar la variable de tipo archivo en la sección

```
Var
   NombreArch:text;
```

Deben utilizarse las siguientes sentencias, de cada uno se describe la sintaxis y su función:

Sentencia	Sintaxis	Función
assign	`assign(nombre archivo Pascal, nombre archivo en el disco)`	Enlaza la variable de tipo text con el archivo físico en el disco. Necesario para lectura/ escritura.
reset	`reset(nombre del archivo Pascal)`	Prepara al archivo para la lectura, ubicándose al comienzo.
read/ readln	`read/readln(nombre archivo Pascal, lista de variables)`	Se leen del archivo los valores y se almacenan en las variables indicadas en la lista.
rewrite	`rewrite(nombre del archivo Pascal)`	Prepara al archivo para la escritura, ubicándose al comienzo.
write/ writeln	`write/writeln(nombre del archivo Pascal, lista de variables )`	Se graban en el archivo los valores contenidos en las variables de la lista.
close	`close(nombre del archivo Pascal)`	Cierra el archivo.
eof	`eof(nombre de archivo Pascal)`	Función booleana que detecta el fin del archivo

Ha de tenerse cuidado con la sentencia rewrite, pues si el archivo no existe, se crea uno nuevo, pero si ya existe, *su contenido se pierde al sobrescribir los nuevos valores*.

E4) A continuación ejemplos utilizando archivos de texto

a) En el archivo Numeros.txt se han grabado datos numéricos de tipo entero. Mostrar y sumar los positivos. Los números han sido grabados en una única línea.

```
program E4a;
 var
   dato, sum:integer;
   Arch:text;{se declara una variable para acceder al archivo de texto}
 Begin
 sum:=0;
 Assign (Arch,'Numeros.txt');{vincula el archivo Numeros.txt con la
                                 variable Arch}
 Reset (Arch);  {prepara el archivo para lectura}
 While not eof(Arch) do {controla que no haya alcanzado el fin de archivo}
        Begin
         Read(Arch,dato); {lee un número del archivo}
         If dato > 0 then
              begin
              sum:= sum + dato;
              Writeln(dato);
              end;
        End;
 Close (Arch); {cierra el archivo}
 Writeln('la suma es', sum);
 End.
```

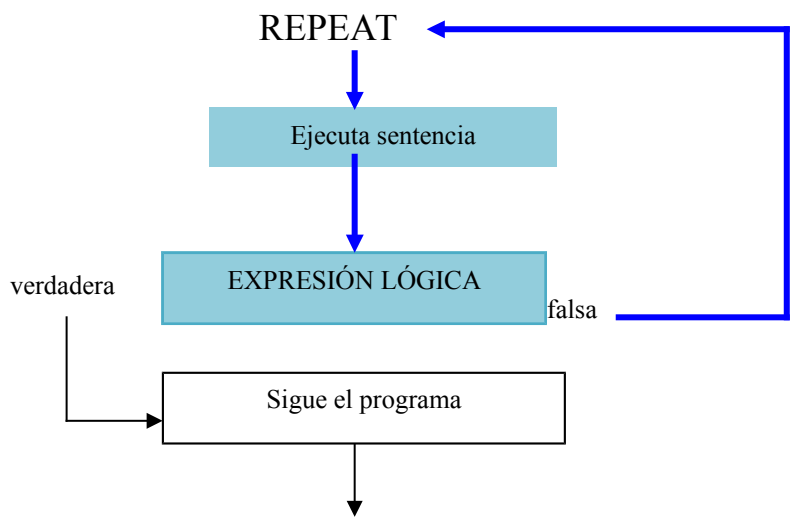
b) En el archivo Clima.txt se ha grabado información climatica, en cada línea:
Fecha (dd/mm/aaa) maxima minima

Se pide leer el archivo y grabar en otro archivo Amplitud.txt las fechas que registren amplitud térmica mayor a 10.

```
program E4b;
 var
   fecha:string[8];
   max, min:real;
   ArchE, ArchS:text;{se declaran un archivo de entrada y otro de salida}
 Begin
 Assign (ArchE,'Clima.txt'); Reset (ArchE);  {prepara el archivo para
lectura}
 Assign (ArchS,'Amplitud.txt'); Rewrite(ArchS);  {prepara el archivo para
                                                  escritura}
 While not eof(ArchE) do {controla que no haya alcanzado el fin de archivo}
     Begin
     Readln(ArchE, fecha, max, min); {lee una medición del archivo}
     If max - min > 10 then
         Writeln(ArchS, fecha);{si la amplitud es mayor a 10 graba la fecha}
     End;
 Close (ArchE); Close (ArchS);{cierra los archivos}
 End.
```

3.Estructura de control: REPEAT-UNTIL

Un caso especial de estructura de control de repetición es la sentencia REPEAT-UNTIL. A diferencia de WHILE-DO, verifica, al final de la estructura si la condición es verdadera o falsa. Si es falsa continúa la ejecución de la sentencia/s del cuerpo del ciclo. En caso contrario, termina.



Sintaxis Pascal:

REPEAT

Sentencia;
UNTIL expresión lógica;

La sentencia (simple o estructurada) entre las palabras claves REPEAT y UNTIL se ejecuta al menos una vez. Como recién vimos, la ejecución repetida se controla mediante la expresión booleana, que es evaluada luego de cada iteración.

E5): Leer un mes (1 .. 12) luego indicar a que trimestre pertenece (1 .. 4) . Validar la lectura hasta que ingrese un mes correcto.

```
program E5;
var
  mes:byte;
Begin
  REPEAT
      writeln('ingrese mes');
      readln(mes);
  UNTIL (1<=mes) and (mes<=12);
  Writeln ('pertenece al trimestre ', (mes-1) div 3 +1)
End.
```

E6) Leer dos números reales, y luego utilizar en forma repetitiva un ***menú*** para obtener uno o más de los siguientes resultados: suma, resta, producto, división, el mayor.

```
program E6;
var
  x, y: real;
  op: char;
Begin
  writeln('ingrese dos números distintos de cero');
  readln(x, y);
  repeat
      writeln('S - Suma');


      writeln('R - Resta');
      writeln('P - Producto');
      writeln('D - Division');
      writeln('M - Mayor');
      writeln('F - Finalizar');
      writeln('Ingrese opcion');
      Readln(op);
      Case op of
      'S':  writeln( x + y :8:2);
      'R':  writeln( x - y :8:2);
      'P':  writeln( x * y :8:2);
      'D':  writeln( x / y :8:2);
      'M': if x > y then
               Writeln(x:8:2)
           Else
               if x < y then
                    Writeln(y:8:2)
               Else
                    Writeln ('son iguales');
       End;
  Until op = 'F';
End.
```

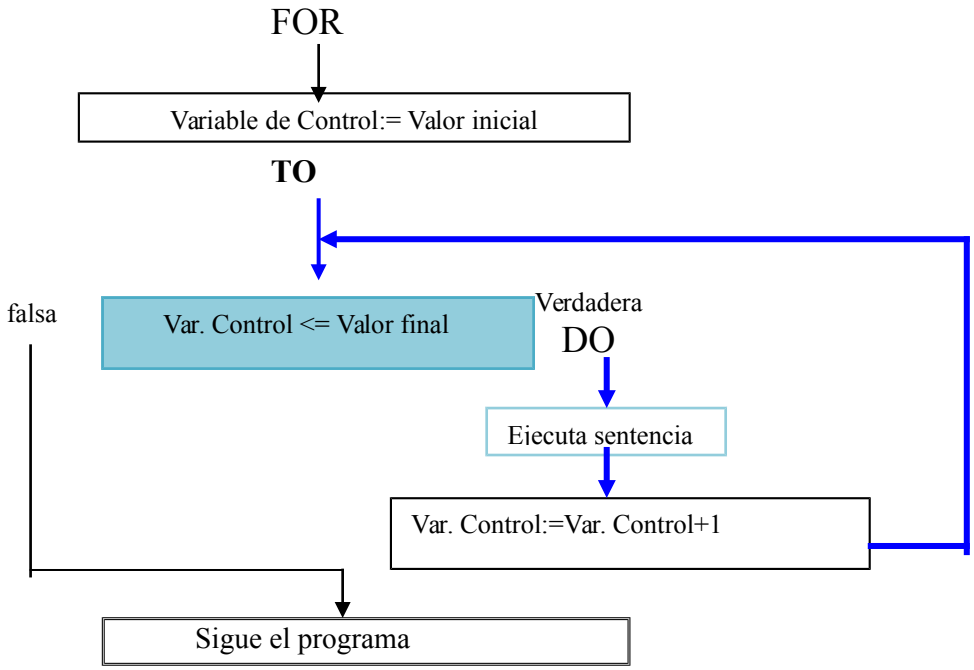
menú

4. Sentencia FOR

La sentencia FOR indica que una sentencia/s debe/n ejecutarse repetidamente mientras una progresión de valores es asignada a la *variable de control* de la sentencia FOR. Se presentan dos casos:

- FOR ascendente (FOR-TO)
- FOR descendente (FOR-DOWNTO)

4.1 FOR ascendente (For to) : valor inicial <= valor final



Sintaxis Pascal:

FOR VC:= VI **TO** VF **DO**
Sentencia;

Donde: VC: es una variable de control.
VI: valor inicial
VF: valor final y VI<=VF

La variable de control, el valor inicial (expresión) y el valor final (expresión), deben ser del mismo tipo ordinal (se excluye los tipos real y string) y NO deben ser alterados por la sentencia FOR. El valor inicial y el valor final se evalúan al comienzo (una sola vez) y si el valor inicial es mayor que el final no se ejecuta el ciclo. La variable de control queda indefinida luego de terminar FOR

E7) Leer un valor K (1..10), luego calcular y mostrar la tabla de multiplicar de K.

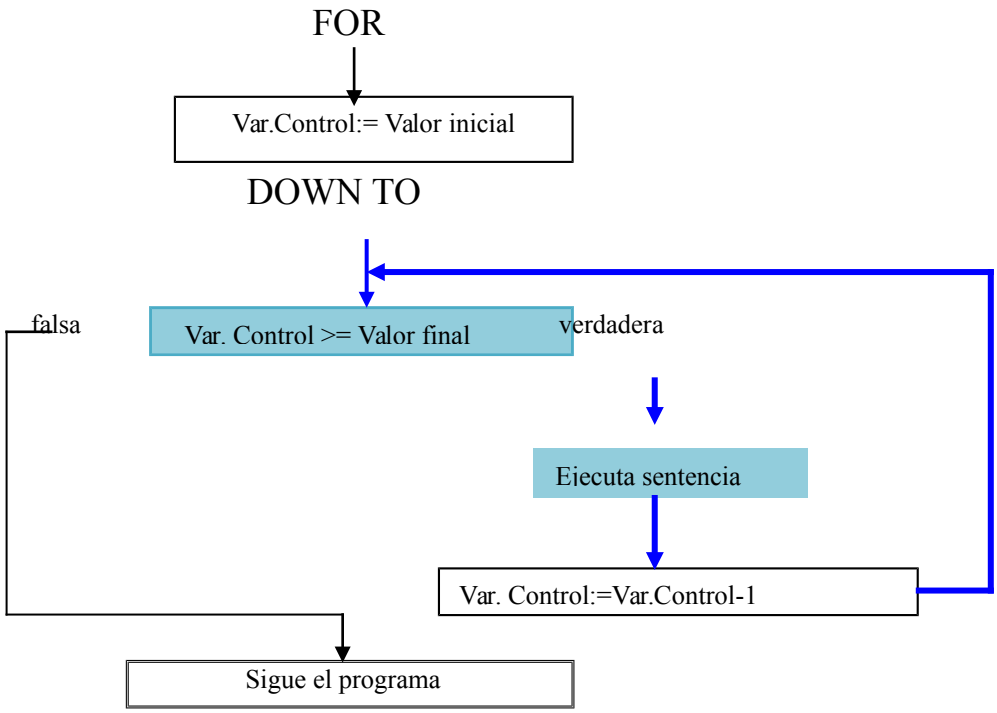
```
Program E7;
 Var
   K,i:integer;
 Begin
  Writeln('Ingrese K');
  Readln(K);
  For i:=1 to 10 do
    writeln( K,' * ', i,' = ', K * i);
 end.
```

Notar que un ciclo For es equivalente a un ciclo While, como muestra el siguiente cuadro:

FOR I:=1 TO N DO
Begin
. . . . . .
. . . . . .
End;
	I:=1; {Inicialización}
	WHILE I<=N DO {Evaluación}
	Begin
	. . . . . .
	. . . . . .
	I:=I+1 {Incremento}
	End;

Por lo tanto siempre puede reemplazarse un For por un While, la inversa no siempre es posible.

4.2 FOR descendente (FOR-DOWNTO) : valor inicial >= valor final



<u>Sintaxis Pascal:</u>

FOR VC:= VI **DOWNTO** VF **DO**
Sentencia;

Donde:
VC: es una variable de control
VI: valor inicial
VF: valor final y VI>=VF

La variable de control, el valor inicial (expresión) y el valor final (expresión), deben ser del mismo tipo ordinal (se excluye los tipos real y string) y NO deben ser alterados por la sentencia FOR. El valor inicial

y el valor final se evalúan al comienzo (una sola vez) y si el valor inicial es menor que el final no se ejecuta el ciclo. La variable de control queda indefinida luego de terminar FOR.

E9) El siguiente programa muestra los números enteros contenidos en un intervalo entero [a,b], en forma descendente

```
program E9;
 var
    i,a,b:integer;
 begin
 Readln(a,b);
 for i:=b downto a do
      write (i:4);
 end.
```

5. Eficiencia algorítmica

El razonamiento algorítmico es una forma especial y potente de nuestro pensamiento. El enfoque algorítmico subyace en el corazón de la ciencia de la computación y representa el "espíritu de la computación" (Harel, 1992).

Debemos tener en cuenta que la ejecución de un algoritmo en una computadora consume **recursos**. Cuando resolvemos un problema resulta útil comparar varios algoritmos para resolverlo. Luego elegir el "mejor" en términos de **espacio** y **tiempo,** i.e., los recursos primarios. Un enfoque para comparar algoritmos que compiten para resolver el mismo problema es elegir aquel que consume **menos** recursos.

El **espacio de memoria** utilizado es medido por elementos tales como número y tamaño de las estructuras de datos usadas o la cantidad de variables incluidas.
El **tiempo de ejecución** puede medirse de diversas formas. Podemos usar un enfoque empírico, mediante la programación y ejecución de algoritmos en una computadora. Los resultados de tales mediciones pueden ser diferentes dependiendo de la computadora usada y en algunos casos puede alentar el mito de que las computadoras son tan increíblemente rápidas y que no existe el problema del tiempo, cosa que es totalmente infundada. Otro enfoque recomendado es una medición universal para determinar el tiempo de ejecución y consiste en *calcular el número de acciones elementales* que hará el procesador cuando ejecuta el algoritmo. Si bien este resultado difiere de entrada en entrada, determina matemáticamente la cantidad de tiempo de ejecución necesaria. La ventaja de esta medición radica en que no depende de una computadora en especial ni en el lenguaje de programación usado y nos permite estudiar la eficiencia de un algoritmo con entradas de cualquier tamaño.

La noción de eficiencia se fundamenta en la teoría de complejidad computacional, que esta fuera del alcance de este curso. Sin embargo, dada la importancia del tema, se presentará a continuación algunos criterios de eficiencia de algoritmos orientados a programadores noveles.

El centro de los criterios se refiere a la medición del número de acciones elementales. Para estudiar estos criterios tratemos de contestar estas preguntas:

a) El programa más corto (en términos del número de líneas), es el más eficiente?
b) Dos programas con el mismo número de líneas, son igualmente eficientes?

Evaluaremos las respuestas mediante ejemplos:

a) El primer problema propuesto establece: calcular e imprimir el producto de dos números enteros positivos, usando solo la operación suma. A continuación se presentan dos programas que lo solucionan:

```
PROGRAM MasLargo;
Var
 a,b,multi,i:integer;
BEGIN
 WRITE('Ingrese dos enteros positivos');
 READLN(a,b);
 multi:=0;
 IF a<b THEN
     FOR i:=1 to a DO
         multi:=multi+b
 ELSE
     FOR i:=1 to b DO
        multi:=multi+a;
 WRITELN('El resultado es =',multi)
END.
```

```
PROGRAM MasCorto;
Var
 a,b,multi,i:integer;
BEGIN
 WRITE('Ingrese dos ent. positivos');
 READLN(a,b);
 multi:=0;
 FOR i:=1 to a DO
     Multi:=Multi+b;
 WRITELN('El resultado es =',multi)
END.
```

Si el criterio de eficiencia sólo contemplara la cantidad de líneas, evidentemente el segundo programa es más eficiente. Pero, si analizamos más detenidamente algunos casos particulares, como, a:=220 y b:=3, el segundo programa ejecuta el ciclo FOR 220 veces (suma 220 veces 3), mientras que el primer programa ejecuta el ciclo 3 veces (suma 3 veces 220). Los casos donde a es menor que b, los dos programas son equivalentes. En términos de tiempo de ejecución, el primero es, en general, más eficiente. Como conclusión tenemos

No siempre el programa más corto (en términos del número de líneas) es más eficiente en tiempo de ejecución

b) Para contestar la segunda pregunta, veamos el siguiente ejemplo donde se presenta dos programas que suman números multiplicados por 100, con igual número de líneas:

```
Program Igual1;
Var
 N,i,numero,suma:integer;
BEGIN
 suma:=0;
 WRITE('Ingrese un numero entero;');
 READLN(N);
 FOR i:=1 TO N DO
   Begin
   WRITE('Ingrese otro num.entero:');
   READLN(numero);
   suma:=suma+numero
   End;
 suma:=suma*100;
 WRITELN('El resultado es : ',suma) END.
```

```
Program Igual2;
Var
 N,i,numero,suma:integer;
BEGIN
 suma:=0;
 WRITE('Ingrese un numero entero;');
 READLN(N);
 FOR i:=1 TO N DO
   Begin
   WRITE('Ingrese otro num.entero:');
   READLN(numero);
   numero:=numero*100;
   suma:=suma+numero
   End;
 WRITELN('El resultado es : ',suma)
END.
```

El segundo programa ha incluido en el ciclo FOR una sentencia (numero:=numero*100) generando de esta manera una operación de multiplicación repetida N veces. Esta operación no es necesaria. Basta ser calculada una vez fuera del ciclo, como la hace el primer programa. Por lo tanto:

Dos programas con el mismo número de líneas, no siempre son igualmente eficientes en tiempos de ejecución

6. EJERCICIOS

1) A continuación se propone generar **archivos de texto** de dos formas diferentes.

a) En la pantalla de **edición de Pascal**, escribir los siguientes datos separados por un espacio: 6 4
b) Generar un archivo guardándolo en la raíz de la unidad C con el nombre DATOS1.txt
c) En un editor de texto plano (por ejemplo : Block de Notas o el editor de texto de Geany) escribir los siguientes datos separados por un espacio: 7 9 1
d) Generar un archivo guardándolo, en la carpeta identificada con su matrícula en la raíz del disco C, con el nombre DATOS2.txt.
e) Realizar la prueba de escritorio del siguiente programa. Luego editarlo y ejecutarlo. El archivo de texto que utilizará el programa es el que fue creado en el ejercicio 1.b)

```
Program LeeDeArchivo;
const
   D=2;
var
    A, B, C : real;
Arch: text;
begin
   Assign(Arch,'C:datos1.txt');
   Reset(Arch);
   Read(Arch,A,B);
   Close(Arch);
   C:=(A-5)*(B-D);
   Write('El resultado es ');
   Writeln(C:6:2);
   Readln;
End.
```

Lote de prueba: 6 , 4

f) Editar, compilar y ejecutar el siguiente programa:

```
Program GrabaEnArchivo;
var
  A,B,C: real;
  Arch: text;
begin
  Assign(Arch,'C:Nuevo.txt');
  Rewrite(Arch);
  Writeln('Ingrese dos números reales');
  Readln(A,B);
  Write(Arch,A);
  Writeln(Arch,B);
  C:=A*B;
  Write(Arch,C:6:2);
  Close(Arch);
  Writeln('terminé');
  Readln;
End.
```

g) Con un editor de texto plano (por ejemplo : Block de Notas o el editor de texto de Geany) abrir el archivo C:Nuevo.txt y visualizar su contenido. Describir los efectos del uso de los procedimientos Write y Writeln al grabar los datos en el archivo.

2) Para los ejercicios a), b) y c) realizar las pruebas de escritorio de los programas, con los lotes de prueba dados en caso de ser necesarios:

a)
```
Program Cuenta;
var
suma, dato, conta:integer;
```

```
begin
  conta:=0; suma:=0;
  while conta< 5 do
    begin
    readln(dato);
    suma:=suma+dato;
    conta:=conta+1;
    end;
  writeln('La suma de ',conta, ' enteros es: ', suma);
  writeln('el promedio de los datos es ',(suma/conta):5:2);
  readln;
end.
```

Lote de prueba : 4 , -4 , 2 , 3 , -2

b)

```
Program potencia;
var
  pot:integer;
begin
  pot:=1;
  repeat
    write(pot:5);
    pot:=pot*2;
  until pot >30;
  readln;
end.
```

c) Datos del archivo 5, 2, 10, -6.

```
Program Maximo;
var
  Dato, Max  :integer;
  Arch : text;
Begin
  Assign(Arch,'Numeros.txt');
  Reset(Arch);
  Max :=-999;
  While Not Eof(Arch) do
    begin
    read(Arch, Dato);
    If Dato>Max then
         Max:=Dato;
    End;
  Close(Arch);
  Writeln(Max);
  readln;
End.
```

3) Se desea calcular el saldo de una cuenta de banco luego de algunos movimientos realizados por el titular de la cuenta. El usuario informará en primer lugar el saldo inicial de la cuenta y luego una serie de datos de la siguiente manera: primero una letra ("D", para depósitos, o "R" para retiros o "F" para fin de datos). En los casos de depósito o retiro también informará el monto correspondiente. Escribir un programa que determine el saldo exacto de la cuenta después de procesar las transacciones. El programa debe validar las letras que se ingresan para indicar la operación bancaria.

4) Considerar el siguiente programa:

```
PROGRAM Cambio;
VAR
   num:INTEGER;
BEGIN
   Write('Ingresa un número entero: ');
   ReadLn(num);

   WHILE num<20 DO
      Begin
      num:=num+1;
      Write(num,' ');
      End;
   ReadLn;
END.
```

a) Reescribir el programa anterior cambiando While por Repeat de manera que ambos produzcan las mismas salidas. ¿Cuál de las dos estructuras de repetición resulta más conveniente?
b) Ejecutar ambos programas, probándolos con los siguientes Lotes de Prueba: 17; 21

5) Escribir un programa que permita al usuario adivinar un número secreto en, a lo sumo, diez intentos. El número secreto es una constante entera aleatoria entre 1 y 50. El usuario ingresa un número, en caso de acertar ganó el juego y el programa termina, sino el programa debe permitirle seguir intentando hasta llegar a diez intentos, en cuyo caso pierde y debe mostrarse la contante que se debía acertar

6) Resolver cada uno de los siguientes problemas utilizando un **MENÚ** que permita optar por una o varias opciones. Incluir una opción para **fin** de programa.
Nota: N y K son números naturales leídos por teclado (variables de entrada).
a) Mostrar por pantalla los números enteros que se encuentran comprendidos en el intervalo [-K , K] en orden ascendente y luego en orden descendente.
b) Mostrar por pantalla los términos de la sucesión 1, 3, 6, 10, 15, 21, 28, ... menores al número K.
c) De un conjunto de números enteros leídos de un archivo, hallar el máximo de los que cumplan con ser divisibles por K.

7) Realizar las pruebas de escritorio del siguiente programa con los lotes de prueba dados

```
Program Promedio;

Var
    Dato, Suma:real;
    N, Num, Cant:integer;
Begin
    readln(N);
    Cant:=0;
    Suma:=0;
    For Num:=1 to N do
      Begin
      readln(Dato);
      If (Dato>0) and (int(Dato)=Dato) then
           Begin
           Cant:=Cant+1;
```

Lotes de prueba: 4, 100.3, 45.8, 320.5, 300.0 ;
3, 100, 45.8, 320

```
          Suma:=Suma+Dato;
          End
        Else
          Writeln(Dato,'No se considera para el cálculo')
      End;
    If Cant<>0 then
      writeln(Suma/Cant)
    else
      Writeln('No se puede efectuar el cálculo');
    readln;
End.
```

8) Leer, desde un archivo de texto, un conjunto de números reales. Se pide calcular el promedio de los números que se encuentran en el intervalo (A , B). A es menor a B, siendo ambos datos de tipo numérico real.

Ejemplo: A = 3.5, B = 6.5
Conjunto = 2, ***4***, ***5.5***, 7, 3.5, ***5.5***, 1.25, 0 → Promedio = ***5***

Reescribir el programa considerando que los números del archivo están ordenados

9) Se tiene un archivo que contiene un texto (una sola línea), leer carácter a carácter el archivo y contar:

- la cantidad de vocales (mayúsculas o minúsculas)
- la cantidad de palabras. Considerar como separadores "**.**" , "**,**" y " ".

<u>Nota</u>: las palabras no tienen más de un separador entre ellas.

10) Realizar la prueba de escritorio del siguiente programa:

```
Program Serie;
Var
  Suma,Signo,N,I,Factorial,K:integer;
Begin
  readln(N);
  Suma:=0;
  Signo:=-1;
  I:=0;
  While I<N do
    Begin
    I:=I+1;
    Factorial:=1;
    For K:=2 to I do
         Factorial:=Factorial*K;
    Suma:=Suma+Factorial*Signo;
    Signo:=-Signo;
    End;
  writeln('El cálculo para ',N,' términos es: ',Suma);
  readln;
End.
```

<u>Lote de prueba</u>: 4

11) Una fábrica de envases plásticos desea evaluar el funcionamiento de sus máquinas. En cada línea de un archivo ha grabado: el número de identificación de la máquina, la cantidad total de envases que produce y la cantidad de envases con fallas. A partir de la información del archivo calcular e indicar:

a) para cada máquina, el número de identificación y el porcentaje de envases desechados respecto al total de envases que ha producido. Junto al cartel DEFICIENTE, NORMAL u OPTIMO según el

porcentaje de envases desechados sea: mayor al 20%, menor o igual al 20% o 0% respectivamente.

b) producción promedio de máquinas con calificación NORMAL.
c) cantidad de máquinas en cada una de las calificaciones.

12) Una prueba consiste en lanzar una moneda hasta que aparezca cara. El número promedio de lanzamientos en una prueba es el límite para n tendiendo a infinito de:

$$\sum_{i=1}^{n}\left(\frac{i}{2^{i}}\right)$$

Encontrar un valor aproximado para este límite (valor de la sumatoria).
Desarrollar dos programas diferentes, teniendo en cuenta las siguientes condiciones:

a) n es dato.
b) El último término de la sumatoria debe ser el menor de los términos que se mantiene mayor que un valor ε dado.

13) Un aeródromo cuenta con 3 tipos de aviones que prestan su servicio diariamente.. Se desea brindar información estadística a la comisión directiva del Aeródromo, sobre la actuación de esas naves en el mes. Se cuenta para ello, con un listado de los distintos vuelos realizados, donde para cada vuelo, figuran los siguientes tres datos:
-Tipo del avión (1 a 3)
-Cantidad de pasajeros transportados
-Altura máxima alcanzada en el vuelo.

Se pide realizar un programa Pascal que, ingresando los datos citados para cada vuelo (se desconoce la cantidad de vuelos realizados en el mes), determine y escriba lo siguiente:

a) El tipo del avión que mayor cantidad de pasajeros transportó en un viaje y la altura que logró en ese vuelo.
b) Los promedios de pasajeros por cada tipo de avión

14) Ingresar desde teclado, para cada uno de los meses del año, las temperaturas máximas y mínimas de cada día (ordenadas por mes). Desarrollar un programa que:

a) para cada uno de los meses muestre por pantalla, la máxima y la mínima absolutas del mes, como así también el día en que se produjeron.
b) muestre por pantalla la temperatura máxima anual, el día y el mes en que se registró.

15)
a) Se desea saber el nombre y el promedio obtenido por el mejor alumno de un curso. Se desconoce la cantidad de alumnos del curso. De cada uno se ingresa: el nombre (25 caracteres), la cantidad de materias aprobadas y, por cada materia, la nota obtenida.

b) Modificar el ejercicio anterior para analizar los datos de 3 cursos, se desea saber:

- el nombre y el promedio del mejor alumno de cada curso
- el nombre y el promedio del mejor alumno de los tres cursos.

EJERCICIOS ADICIONALES

16) En una fábrica de heladeras se realiza un control de calidad, sobre una cantidad no conocida de unidades, identificadas cada una con un código. Cada unidad es sometida a N pruebas de calidad (N es igual para todas las unidades). El resultado de cada prueba puede ser excelente, bueno, regular o malo. Escribir un programa Pascal para:
a) Ingresar para cada producto su código ('zzz' indica fin de datos) y a continuación los N resultados de los controles efectuados, codificados como [E]xcelente, [B]ueno, [R]egular, [M]alo.
b) Determinar y escribir si el producto es "aceptado" o "rechazado". Son aceptados los productos que cumplen con lo siguiente: No tienen ninguna prueba con calificación mala (M) y tienen como mínimo un 50% entre excelentes y buenos (E o B).
c) Escribir el porcentaje de productos "rechazados".

17) Una empresa tecnológica solicita contraseña para el acceso a sus sistemas. La misma debe estar conformada como mínimo por 8 caracteres, incluyendo obligatoriamente, *al menos una* letra y *exactamente* cuatro dígitos. No están permitidos otro tipo de caracteres. Para finalizar el ingreso de cada contraseña se usa un * que no se considera como parte de ella).

Se pide desarrollar un programa Pascal, correctamente indentado y eficiente, que ingrese (carácter a carácter) N contraseñas de longitud variable, evalúe e indique la validez de cada una, y al final informe la longitud de la contraseña inválida más larga (puede no existir).

Ejemplo N=7

Si las contraseñas ingresadas fueran:

eR68G12a*	válida
91jY643ebjp*	inválida
eRty74kLh*	inválida
24fG92*	inválida
aj85gt32*	válida
eL8j$8215*	inválida
dGb9357jKoup2*	válida

La salida del programa sería: La longitud de la contraseña inválida más larga es de 11 caracteres